

Scalable Memory-Centric Data Systems for Modern Datacenters

¹ Sai Sowjanya Edupuganti

Dept. of Computer Science

University of North Texas

Denton, TX, USA

SaiSowjanyaEdupuganti@my.unt.edu

² Anusha Pujari

Dept. of Computer Science

University of North Texas

Denton, TX, USA

AnushaPujari@my.unt.edu

³ Lavanya Vijinigiri

Dept. of Computer Engineering

University of North Texas

Denton, TX, USA

LavanyaVijinigiri@my.unt.edu

⁴ Sai Vijay Sankar Bheema

Dept. of Computer Engineering

University of North Texas

Denton, TX, USA

SaiVijaySankarBheema@my.unt.edu

Abstract—The requirements in terms of datacenter applications today are that the data be fast to access, the operations be concurrent, and that the memory be used effectively. Some examples of these types of systems are online database management, cloud computing, and big data processing. In order for them to serve well, they have to be capable of handling millions of requests at a very low latency. Although the RAM can work faster than storage capacity, just storing the data in memory alone does not help solve all the scalability concerns. Contention, synchronization, and even communication may happen when the indexes are handled.

This paper reviews three OSDI 2023 papers that address scalability challenges at different system layers. ScaleDB suggests a scalable in-memory database using asynchrony, which enables a greater separation of transaction execution and range data indexing; thereby minimizing range-index contention. The initial supporting article explains how to enhance the replication properties of persistent memory-based key-value store implementations by use of efficient use of remote direct memory access (RDMA). The SMART paper explains how adaptive radix trees can be redesigned for disaggregated memory systems. And subsequently enable the adaptation of such trees based on replication. Taken together, these three papers have shown that the performance of the current systems is not only a factor of enhanced hardware, but also the proper design of systems in general. This report has summarized each of the articles and outlined the key approaches employed, strengths/weaknesses, and future trends of scalable memory-based systems in datacenters.

Index Terms—in-memory database, persistent memory, RDMA, disaggregated memory, scalability, concurrency control, datacenter systems

I. INTRODUCTION

Today, applications are becoming increasingly memory-intensive. Applications such as cloud computing services, social media networks, recommendation engines, banking applications, and analytics systems require processing of huge volumes of information with a negligible latency factor. Thus, most applications nowadays prefer storing relevant information in main memory instead of only utilizing hard drive-based storage space. While in-memory systems offer fast response

times, concurrent access by numerous users/threads presents several technical difficulties for developers.

Although it seems obvious that increasing memory speeds will improve system efficiency, this does not necessarily happen in practice. The factors such as contention among the indexes, coordination among transactions, overhead costs of replicating and accessing remote memory can reduce efficiency of the system even when the data is stored in memory. These kinds of problems become particularly significant for data centers, which have to work in multiple machines and provide services to a great number of customers.

There are three papers on scalable memory-centric data systems in OSDI 2023. This review will mainly focus on *ScaleDB: An Asynchronous, Scalable In-Memory Database* first, which will improve scalability and minimize contention on range indexes for in-memory databases. Second, the paper *Replication of Persistent Memory Key-Value Stores Using Efficient RDMA Abstraction* discusses how to replicate persistent memory K/V stores utilizing RDMA as the replication mechanism. Finally, *SMART: An Adaptive Radix Tree for Disaggregated Memory* examines the design of a high-performance data structure for disaggregated memory..

These documents complement one another because they all approach related challenges from different levels within a system. For example, ScaleDB's focus is on transactional processing within databases; the RDMA paper discusses remote memory access through replication/communication; and SMART addresses issues related to data structure design pertaining to remote memory access. All together, they provide a more complete understanding of potential solutions therefore reducing bottlenecks for modern datacenter systems to optimize system performance.

The primary purpose of this report is to provide a clear understanding of the concepts presented in these research papers. This survey report will not just present the results of the paper, but will rather discuss the motivation, design

decisions, advantages and disadvantages of the research work. The primary theme of this report is that scalability in data centers requires reduction of contention and unneeded data transfer.

II. BACKGROUND AND MOTIVATION

A. Importance of Memory-Centric Architectures

In traditional systems, storage devices such as hard drives and SSDs were often the main performance bottleneck. Databases had to carefully reduce disk access because reading from disk was much slower than reading from memory. As memory capacity increased and became cheaper, many systems started storing most of their working data in memory. This led to the popularity of in-memory databases and memory-based key-value stores.

However, memory-centric systems have their own problems. When many CPU cores access the same memory structure, they may compete for locks, cache lines, or index nodes. This can reduce scalability even if the raw memory access time is low. A system may perform well with a small number of threads but stop improving when more threads are added. This is one of the main problems addressed by ScaleDB.

B. Persistent Memory and Replication

Another useful technology is persistent memory. It gives access speed close to ordinary memory but keeps the data even when there is a power outage. Persistent memory is useful for applications requiring both speed and persistence, such as databases and key-value stores. However, data replication is needed when data needs to be kept durable as a result. Replication can increase the overhead because data needs to be transferred to backup machines.

RDMA (remote direct memory access) tries to help in solving that problem. With RDMA, it becomes possible to access memory from other machines without using their CPUs much. The first paper investigates that problem.

C. Disaggregated Memory

Disaggregated memory is a relatively new data center architecture where computational units and memory units are independent. Unlike traditional architectures where each computer has its own dedicated memory, memory can reside in memory pools at different servers and can be distributed among various computers. This will increase efficiency since unused memory on one computer can be utilized on other computers.

However, remote memory access is slower than local memory access. Data structures optimized for local memory access will not be efficient if accessing each memory requires network calls. To solve this problem, SMART designs the adaptive radix tree for disaggregated memory.

III. SELECTED PAPERS

A. Main Paper: ScaleDB

ScaleDB is a serializable in-memory transactional database. It aims to address the scalability of transactions using range

indexes. Range indexes are crucial since they facilitate queries such as retrieving records within a specified range. A typical use case would be locating all customer entries whose IDs fall within a specified interval or retrieving orders within a specific time range.

However, the downside to range indexes is that they could lead to contention issues. For instance, each transaction might have to directly update the range index. In such scenarios, conflicts arise when multiple threads concurrently access the same memory location. The conflict worsens with an increase in the number of threads. Notably, the developers of ScaleDB observe that some index updates could be postponed. As a result, they propose a solution that performs range-index updates asynchronously.

In essence, the key idea behind the design is to separate the execution of a transaction from partial indexing operations. This means that each transaction can commit changes and terminate without performing index operations on all index structures. Subsequently, background threads perform the indexing operations. It is worth noting that separating the processes reduces direct contention on the index and enables parallelism.

The key difficulty faced by such designs is ensuring correctness. More specifically, databases have to ensure that transactions are serializable. Although the updates are not immediately applied, the system should still be able to detect any interference to guarantee consistency. In this regard, ScaleDB employs asynchronous concurrency control and phantom detection.

B. Supporting Paper 1: RDMA Replication for Persistent Memory

The first paper focuses on persistent memory key-value stores. Key-value stores are popular in data centers because they offer easy and quick access to data using keys. Persistent memory makes such databases faster and durable, but still, replication is costly.

In particular, replication is important since any single machine may fail. In this case, a machine holding a copy of data would cause data loss. Thus, replication should be done for persisting memory systems to have a backup of the same data on other machines. Classical replication requires some efforts like CPU involvement, use of software queues, logging, and copying data multiple times, which increases latency.

This paper introduces a new abstraction called RDMA which allows writing data from a primary machine to memory of a backup machine without CPU overhead. Thus, the paper concentrates on replication efficiency but tries not to make systems more complicated to maintain.

This paper is close to ScaleDB in its motivation since both papers concentrate on reducing the overhead in memory-based systems. ScaleDB reduces the index contention within a database, whereas RDMA reduces communication overhead.

C. Supporting Paper 2: SMART

SMART targets disaggregated memory. In this scheme, compute nodes have access to the memory which may be on a

remote memory node. An ordinary tree structure will involve traversing several pointers. In case each traversal results in accessing memory from another node, the process may take too long.

SMART reinvents the adaptive radix tree for this purpose. Adaptive radix trees are beneficial in that they allow for faster lookups and updates. However, the original implementation is designed for local memory. The researchers have thus restructured and repurposed the tree to limit remote accesses.

Some of the novel techniques proposed in the paper include read delegation and write combining. Read delegation allows certain read operations to be executed with minimal cost. Write combining enables clustering of multiple writes to increase efficiency. Both techniques contribute to improved latency and throughput by SMART.

The paper has a good connection with ScaleDB. It is clear that the two papers concern themselves with indexes. ScaleDB analyzes the impact of range indexes on in-memory databases. On the other hand, SMART evaluates the impact of tree structures on disaggregated memory.

IV. TECHNIQUES AND METHODOLOGY

A. Asynchronous Range Index Updates in ScaleDB

ScaleDB has an asynchronous range index update method. In a classic in-memory database, indexes are updated on-the-fly when executing transactions. Although this is a simple method, it may create contention when there are several transactions that are trying to update the same index at the same time resulting in blocking.

ScaleDB enhances this design by delaying some index updates. The system does not update the range index every transaction and uses the changes to update at a later time. These accrue updates are batched and merged into the index by background workers. This saves time that the transactions would have used to read common index structures.

The benefit of this method is that it takes the index maintenance out of critical path of transactions to execute and transactions can be done quicker with the index being updated by background processes. But it is difficult to be sure. ScaleDB addresses this with the help of concurrency control and phantom detection mechanisms.

B. Asynchronous Concurrency Control

Concurrency control dictates whether transactions can run in parallel. In ScaleDB, a concurrent control process is used asynchronously, wherein transactions occur independently while updates to the indices are made independently.

Before making the transaction commit, the system will check if the transaction remains valid. It is crucial because, in range queries, there could be an anomaly called phantoms. The phantom phenomenon refers to a situation in which a transaction accesses a series of records but another transaction adds new records within the range, causing inconsistency.

Even with delays in update processes, the system will detect these anomalies and validate the transaction. The benefit of this mechanism is that it provides performance gains without

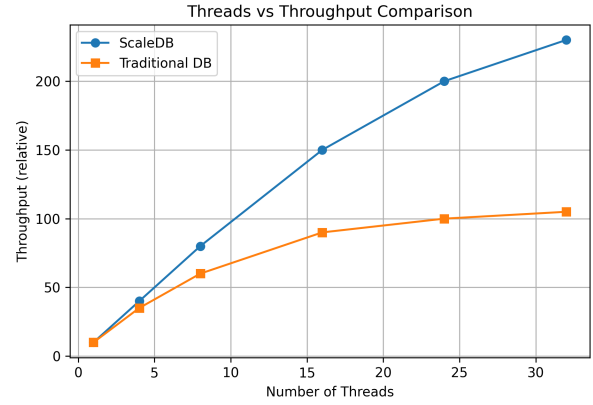


Fig. 1. Threads vs Throughput comparison between ScaleDB and traditional systems. The values are illustrative and reflect scalability trends discussed in the paper.

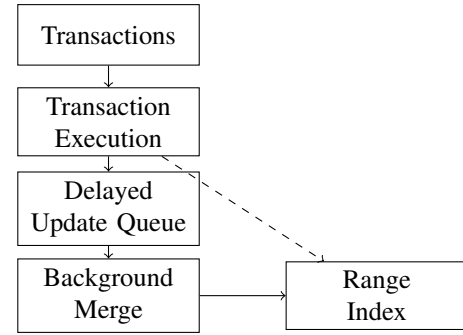


Fig. 2. Overview of ScaleDB asynchronous index updates.

perplexing strong consistency. The drawbacks of this technique is maximized system difficulty of operation.

C. RDMA-Based Replication

A primary node usually communicates the respective data to a backup node via multiple layers of software during traditional data replication. Traditional replication is often subject to latency and overhead, as both primary and backup nodes need to undergo CPU processing of the replicated data.

The use of RDMA allows the primary node to write directly into the memory of the backup node, eliminating the need for CPU processing or making additional copies of the replicated data.

Communication through RDMA has improved greatly and therefore has provided significantly quicker latency and a higher efficiency for replication. Together, these two factors lead to increased system quality for persistent memory systems (durable memory) because of the need for quick and consistent replication.

D. SMART Radix Tree for Disaggregated Memory

The SMART radix tree is an adaptation of the adaptive radix tree (ART) that has been designed for disaggregated memory systems. In a conventional radix tree there are many pointer

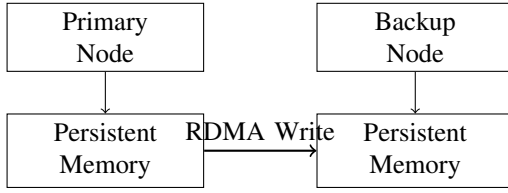


Fig. 3. Simplified RDMA replication between persistent memory nodes.

dereference during the lookup process, which provide a fast and efficient manner to access local memory, but are costly in terms of remote memory access due to the high latency associated with these types of accesses.

The SMART radix tree reduces the remote overhead associated with lookups by redesigning the radix tree structure to minimize the number of accesses to remote memory location. In addition, the SMART radix tree utilizes read delegation and write combining techniques to further reduce the number of remote memory accesses. Read delegation helps eliminate the number of remote read operations by delegating them, while write combining allows multiple writes to be sent in one message to the remote location for execution.

Collectively these optimization strategies greatly improve the performance of disaggregated memory systems by minimizing the number of costly remote operations encountered.

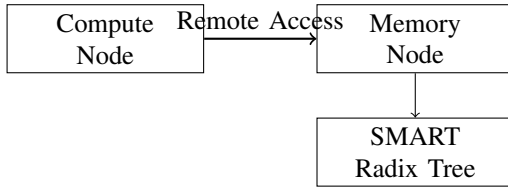


Fig. 4. Overview of SMART in a disaggregated memory system.

V. COMPARISON AND DISCUSSION

A key difference among these papers is the system level at which each one solves the scalability problem. ScaleDB works at the transaction-processing level by reducing contention on range indexes. The RDMA replication paper works at the communication and durability level by reducing the cost of sending updates to backup machines. SMART works at the data-structure level by redesigning tree traversal for remote memory. This comparison shows that memory-system bottlenecks do not come from memory speed alone; they can also come from synchronization, replication, and index design.

ScaleDB is mainly a database system paper. Its strength is that it attacks a practical bottleneck in in-memory databases. The system improves scalability by moving some index work away from transaction execution. However, this design increases complexity because correctness must be preserved while updates are delayed.

The RDMA paper is mainly a storage and replication paper. Its strength is that it takes advantage of modern networking hardware to reduce replication cost. This is useful for persistent memory systems where durability and availability are

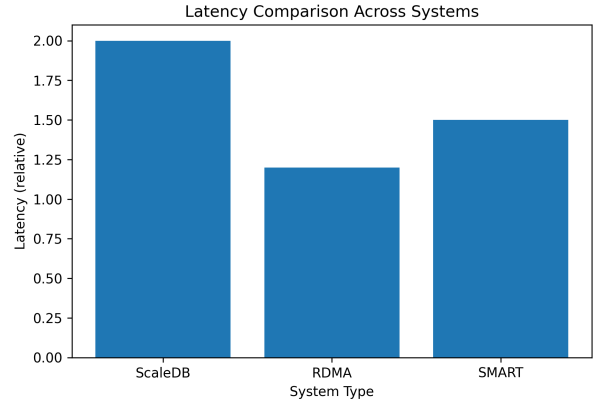


Fig. 5. Latency comparison across ScaleDB, RDMA-based replication, and SMART systems. The values are illustrative and highlight differences in system efficiency.

important. Its limitation is that it depends on RDMA-capable hardware and careful management of remote memory writes.

SMART is mainly a data structure and memory systems paper. Its strength is that it redesigns a common index structure for disaggregated memory. This is important because many old data structures were designed for local memory and may not work well when memory is remote. Its limitation is that the design is specialized for disaggregated memory environments.

TABLE I
COMPARISON OF SELECTED PAPERS

Paper	Main Idea	Main Limitation
ScaleDB	Delays range-index updates to reduce transaction contention	More complex correctness and commit logic
RDMA Replication	Uses RDMA to speed up persistent memory replication	Requires RDMA hardware support
SMART	Redesigns radix tree for disaggregated memory	Most useful in remote memory systems

VI. STRENGTHS AND WEAKNESSES

ScaleDB has a strong practical motivation because in-memory databases are widely used. Its main strength is that it improves scalability without simply relying on faster hardware. It changes the way index updates are handled. A weakness is that asynchronous behavior makes the system harder to understand and implement.

The RDMA replication paper is strong because it addresses a real bottleneck in persistent memory systems. Replication is necessary for reliability, but it can slow down the system. RDMA helps reduce this cost. However, the approach may be harder to deploy in environments that do not already support RDMA.

SMART is strong because it recognizes that disaggregated memory needs new data structure designs. It does not simply reuse a local-memory tree. Instead, it redesigns the tree for

remote access. A weakness is that the system is tied to a specific architecture and may not be as useful for traditional systems.

VII. CHALLENGES AND FUTURE WORK

One of the problems that occurs with all three systems is complexity. Memory today is not a simple topic. When the performance of one layer is increased, the complexity of another layer is increased to achieve this performance gain. For instance, in ScaleDB, although there is improvement in the throughput of operations, detection of conflicts is made more complex. In RDMA, although there is faster replication, handling of remote memory becomes difficult.

Another challenge with deployment is that while research systems may have strong experimental results, real datacenters operate under many different workloads and hardware configurations, so a design that works well in one particular environment may not work as well in another, and would require modifications to the design. Future work should focus on helping to increase the ease with which these techniques can be integrated into already-deployed systems.

A direction for an in-memory database is to combine all three concepts: use asynchronous indexing methods (like ScaleDB), replicate data asynchronously over RDMA for durable storage, and create remote memory-aware indexes (like SMART). This type of architecture can minimize contention, reduce the amount of time spent replicating data, and leverage disaggregated memory.

Energy efficiency is another possible research direction. Datacenters consume a large amount of power. If memory-centric systems reduce unnecessary data movement, they may also reduce energy usage. Future research could study not only throughput and latency, but also energy cost.

VIII. CONCLUSION

The goal of this survey is to analyze three papers published at OSDI 2023 that are all focused on scalable memory-centered data systems. The primary paper of the three, ScaleDB, presents a novel way to increase the scalability of in-memory databases by utilizing asynchronous updates to their range indices. The first supporting paper enhances the Key-Value Store's replication to persistent memory through the use of RDMA. The second supporting paper provides performance improvements for indexing in disaggregated memory environments through new algorithms.

However, the key takeaway from the above discussion would be that simply having fast memory is not enough to solve issues related to performance in data centers in the current environment. There should be improvements in concurrency, replication and data structures as well to make systems better. The above three papers address different aspects of this.

In conclusion, each of these works demonstrate that, for the future of datacenter systems, reducing contention and unnecessary data movement will be key. A complete future system will most likely utilize elements of all three approaches to achieve the goals of scalable, reliable, and efficient data processing.

REFERENCES

- [1] S. A. Mehdi, D. Hwang, S. Peter, and L. Alvisi, "ScaleDB: A Scalable, Asynchronous In-Memory Database," in *Proc. 17th USENIX Symposium on Operating Systems Design and Implementation*, 2023.
- [2] Q. Wang, Y. Lu, J. Wang, and J. Shu, "Replicating Persistent Memory Key-Value Stores with Efficient RDMA Abstraction," in *Proc. 17th USENIX Symposium on Operating Systems Design and Implementation*, 2023.
- [3] X. Luo, P. Zuo, J. Shen, J. Gu, X. Wang, M. R. Lyu, and Y. Zhou, "SMART: A High-Performance Adaptive Radix Tree for Disaggregated Memory," in *Proc. 17th USENIX Symposium on Operating Systems Design and Implementation*, 2023.